# Q10: Why Do Models Solve Hard Problems Yet Make Simple Mistakes?

DLS 2026 — module author: Kukla

October 2026

## Learning objectives

- Explain "jagged" capability profiles and why they differ from human competence curves.
- Trace specific failure modes to the training objective and tokenization.
- Predict, roughly, where a model will be unexpectedly strong or weak.

> **Companion video (6:35):** *The Strawberry Problem: why LLMs can pass tests but cannot count letters* — Arena AI.
> https://www.youtube.com/watch?v=HP3vKcufYCY

## 1 The puzzle

Student response #18 captures a common shock: a model writes a correct merge-sort or explains general relativity, then miscounts the letters in "strawberry" or botches $7 \times 8$. For a human, competence is *correlated*—someone who can do the hard thing can almost always do the easy thing. For LLMs it is not. This is the **jagged frontier**: capability varies sharply and unpredictably across tasks that look similarly easy to us [1]. Understanding why dissolves the paradox.

## 2 Root cause 1: the training objective

A model is trained to predict the next token on its corpus, not to be uniformly competent. So capability tracks *what the training distribution rewarded*, not human intuitions about difficulty. Explaining relativity is *easy* for the model because the internet contains vast, redundant, well-structured prose about it; precise multi-digit arithmetic is *hard* because correct long calculations are comparatively rare and the model was never optimized to run an algorithm, only to continue text. "Hard" and "easy" are defined by data frequency and the objective, not by logical complexity.

## 3 Root cause 2: tokenization

Several classic "simple" failures are tokenization artifacts. The model does not see letters; it sees sub-word tokens (module on Building a Transformer). "Strawberry" may be a few tokens with no explicit letter inventory, so counting r's requires reconstructing spelling the model never represents directly. Likewise, numbers are split into chunks that break positional arithmetic. These are not "stupidity"— they are predictable consequences of the input representation, and they vanish when

the task is reframed to expose the needed structure (spell it out, use digits with separators, or give a tool).

## 4 Root cause 3: one forward pass, no built-in checking

Without chain-of-thought or tools, the model commits to each token in a single pass with no scratchpad and no verification step. Humans catch their own arithmetic slips by re-checking; a base model has no such loop unless prompted to reason (module Q05) or given a calculator. So errors that a quick self-check would catch slip through. This is why the *same* model often fixes its "dumb" mistake the instant you ask it to think step by step—the capability was latent; the harness was missing.

## 5 A predictive heuristic

You can anticipate jaggedness. Expect **strength** where the task is well-represented in text, tolerant of small errors, and pattern-like. Expect **weakness** where it requires exact symbol manipulation, long error-free chains, information below the token level (characters, digits), or knowledge rare in the corpus. The companion code (`code/jagged_demo.py`) illustrates the tokenization angle with a character-counting example and shows how reframing removes the error.

## 6 A gallery of predictable failures

The three root causes let us explain a whole gallery of "how is it this dumb" moments without surprise. *Letter counting and spelling* fail because tokens hide characters. *Multi-digit multiplication* fails because digits are chunked and the model was never trained to execute an algorithm, only to continue plausible text. *Reversing a string* or counting words fails for the same sub-token reason. *Rare factual lookups* fail because the fact was infrequent in the corpus, even when adjacent, more common facts are solid. *Consistency over long outputs* fails because there is no global planner enforcing coherence token to token. In every case the failure is not random—given the objective, the tokenizer, and the single-pass constraint, it is *predictable in advance*. That predictability is the useful part: it turns "the model is unreliable" into "the model is unreliable *here, for this reason*, and reliable *there*."

## 7 Contrast with human error

A human who can integrate by parts will essentially never miscount the letters in a short word; their competence is anchored to understanding. The model's competence is anchored to *statistics of text*, so the two error surfaces simply do not line up. This is why our intuitions mislead us so badly when judging models: we unconsciously use "can do the hard thing" as a proxy for "can do the easy thing," a proxy that is valid for people and invalid for LLMs. Recognizing that the two systems have different competence geometries is the single most useful mental adjustment a new practitioner can make.

## 8 Why this matters

Jaggedness means you cannot infer a model's reliability on task B from its brilliance on task A. For deployment this is central: pair models with verifiers and tools for the brittle regimes (arithmetic, counting, exact retrieval), and do not trust "it's smart, so it must get this right." The evaluation track develops how to map a model's frontier rather than assuming a single competence level.

## 9 Summary

Models make simple mistakes because "simple" is a human notion; the model's competence tracks training frequency and the next-token objective, is distorted by tokenization, and lacks a built-in checking loop. The capability profile is jagged and must be *measured*, not assumed—and brittle regimes are best handled with reasoning prompts, tools, and verifiers.

## References

[1] Fabrizio Dell'Acqua, Edward McFowland, Ethan R. Mollick, et al. Navigating the jagged technological frontier: Field experimental evidence of the effects of ai on knowledge worker productivity and quality. *Harvard Business School Working Paper 24-013*, 2023.